

MYSQL VIEWS

Practical Examples and Student Exercises using MySQL Workbench

1. What is a View?

A **View** is a **virtual table** created from one or more existing tables.

- It **does not store data** itself.
- It stores only the **SQL query**.
- Whenever you query the view, MySQL retrieves the latest data from the underlying tables.

2. Setup: Employees and Departments Tables

Run this setup first. If these tables already exist with data, you may skip the setup.

```
CREATE DATABASE IF NOT EXISTS employeedb1;  
USE employeedb1;
```

```
CREATE TABLE IF NOT EXISTS departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
CREATE TABLE IF NOT EXISTS employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    salary DECIMAL(10,2)  
);
```

```
INSERT IGNORE INTO departments VALUES  
(1,'CSE'),  
(2,'ECE'),  
(3,'EEE');
```

```
INSERT IGNORE INTO employees VALUES  
(101,'Rahul',1,50000),  
(102,'Priya',2,65000),  
(103,'Anil',1,55000),  
(104,'Sneha',3,70000),  
(105,'Kiran',2,48000);
```

3. Verify the Tables

SHOW TABLES;

SELECT * FROM departments;

SELECT * FROM employees;

OUTPUT:

The screenshot displays a database management interface with a query editor and a results pane. The query editor shows the following SQL statements:

```
22 (102, 'Priya', 2, 65000),
23 (103, 'Anil', 1, 55000),
24 (104, 'Sneha', 3, 70000),
25 (105, 'Kiran', 2, 48000);
26 • SHOW TABLES;
27
28
```

The results pane shows the output of the `SHOW TABLES;` query, listing the tables in the `companydb1` database:

Tables_in_companydb1
departments
employees

Below the results pane, the "Output" section shows the execution log for the queries:

#	Time	Action	Message
7	09:09:00	CREATE TABLE IF NOT EXISTS departments (dept_id INT PRIMARY KEY, dept_name VARCHAR(50))	0 row(s) affected
8	09:09:36	INSERT IGNORE INTO departments VALUES (1,'CSE'), (2,'ECE'), (3,'EEE')	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
9	09:10:08	INSERT IGNORE INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,'Anil',1,55000), ...	Error Code: 1146. Table 'companydb1.employees' doesn't exist
10	09:11:25	CREATE TABLE IF NOT EXISTS employees (emp_id INT PRIMARY KEY, emp_name VARCHAR(50), ...	0 row(s) affected
11	09:11:47	INSERT IGNORE INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,'Anil',1,55000), ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
12	09:12:23	SHOW TABLES	2 row(s) returned

Limit to 1000 rows

```

22      (102, 'Priya', 2, 65000),
23      (103, 'Anil', 1, 55000),
24      (104, 'Sneha', 3, 70000),
25      (105, 'Kiran', 2, 48000);
26 •    SHOW TABLES;
27 •    SELECT * FROM departments;
28

```

Result Grid

	dept_id	dept_name
▶	1	CSE
	2	ECE
	3	EEE
*	NULL	NULL

Limit to 1000 rows

```

23      (103, 'Anil', 1, 55000),
24      (104, 'Sneha', 3, 70000),
25      (105, 'Kiran', 2, 48000);
26 •    SHOW TABLES;
27 •    SELECT * FROM departments;
28 •    SELECT * FROM employees;
29

```

Result Grid

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	50000.00
	102	Priya	2	65000.00
	103	Anil	1	55000.00
	104	Sneha	3	70000.00
	105	Kiran	2	48000.00
*	NULL	NULL	NULL	NULL

View 1 – Display All Employees

Question: Create a View named EmployeeView to display all columns from employees.

DROP VIEW IF EXISTS EmployeeView;

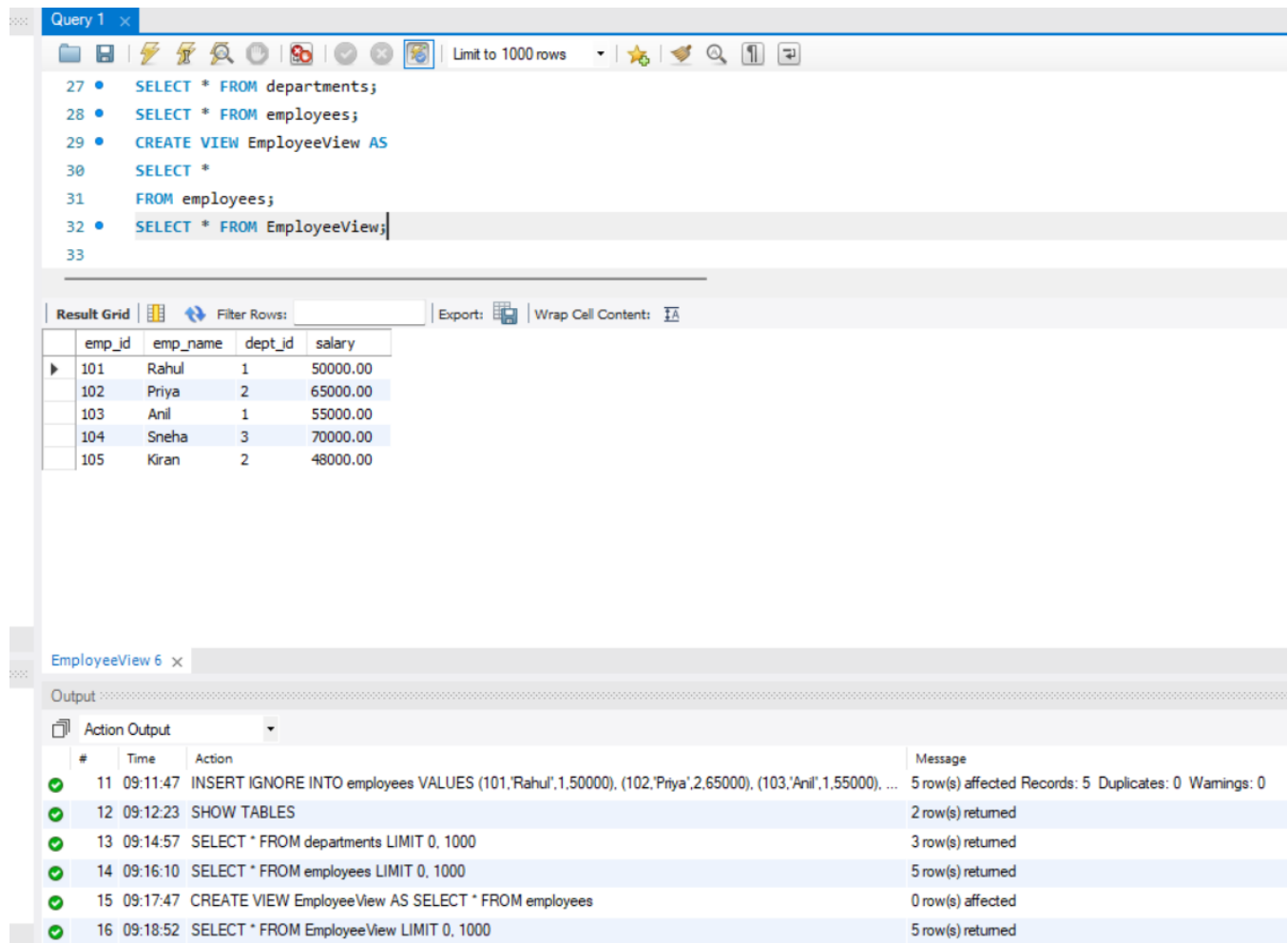
CREATE VIEW EmployeeView AS

SELECT *

FROM employees;

SELECT * FROM EmployeeView;

OUTPUT:



The screenshot displays a SQL IDE interface. The top pane, titled 'Query 1', contains the following SQL script:

```
27 • SELECT * FROM departments;
28 • SELECT * FROM employees;
29 • CREATE VIEW EmployeeView AS
30   SELECT *
31   FROM employees;
32 • SELECT * FROM EmployeeView;
33
```

The bottom pane, titled 'EmployeeView 6', shows the 'Output' window with the 'Action Output' tab selected. It displays a log of SQL actions and their results:

#	Time	Action	Message
✓ 11	09:11:47	INSERT IGNORE INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,'Anil',1,55000), ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
✓ 12	09:12:23	SHOW TABLES	2 row(s) returned
✓ 13	09:14:57	SELECT * FROM departments LIMIT 0, 1000	3 row(s) returned
✓ 14	09:16:10	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned
✓ 15	09:17:47	CREATE VIEW EmployeeView AS SELECT * FROM employees	0 row(s) affected
✓ 16	09:18:52	SELECT * FROM EmployeeView LIMIT 0, 1000	5 row(s) returned

Below the log, a 'Result Grid' is visible, showing the data returned by the final query (SELECT * FROM EmployeeView):

emp_id	emp_name	dept_id	salary
101	Rahul	1	50000.00
102	Priya	2	65000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00
105	Kiran	2	48000.00

View 2 – Selected Columns

Question: Create a View showing employee ID, employee name and salary only.

DROP VIEW IF EXISTS EmployeeBasicView;

CREATE VIEW EmployeeBasicView AS

SELECT emp_id, emp_name, salary

FROM employees;

SELECT * FROM EmployeeBasicView;

OUTPUT:

The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a query window titled 'Query 1' with the following SQL script:

```
32 • SELECT * FROM EmployeeView;
33 • CREATE VIEW EmployeeBasicView AS
34   SELECT emp_id, emp_name, salary
35   FROM employees;
36
37 • SELECT * FROM EmployeeBasicView;
38
```

The bottom pane shows the 'Result Grid' with the following data:

emp_id	emp_name	salary
101	Rahul	50000.00
102	Priya	65000.00
103	Anil	55000.00
104	Sneha	70000.00
105	Kiran	48000.00

Below the result grid, the 'Output' pane shows the 'Action Output' for the query execution:

#	Time	Action	Message
✓ 13	09:14:57	SELECT * FROM departments LIMIT 0, 1000	3 row(s) returned
✓ 14	09:16:10	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned
✓ 15	09:17:47	CREATE VIEW EmployeeView AS SELECT * FROM employees	0 row(s) affected
✓ 16	09:18:52	SELECT * FROM EmployeeView LIMIT 0, 1000	5 row(s) returned
✓ 17	09:24:28	CREATE VIEW EmployeeBasicView AS SELECT emp_id, emp_name, salary FROM employees	0 row(s) affected
✓ 18	09:24:35	SELECT * FROM EmployeeBasicView LIMIT 0, 1000	5 row(s) returned

View 3 – High Salary Employees

Question: Create a View that displays employees earning more than 55000.

DROP VIEW IF EXISTS HighSalaryEmployees;

```
CREATE VIEW HighSalaryEmployees AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary > 55000;
```

SELECT * FROM HighSalaryEmployees;

Output:

The screenshot displays a database query editor with two tabs: 'Query 1' and 'HighSalaryEmployees2'. The 'Query 1' tab contains the following SQL code:

```
3 • CREATE VIEW HighSalaryEmployees AS
4   SELECT emp_id, emp_name, salary
5   FROM employees
6   WHERE salary > 55000;
7
8 • SELECT * FROM HighSalaryEmployees;
9
```

Below the query editor, the 'Result Grid' shows the output of the second query. It contains two rows of data:

emp_id	emp_name	salary
102	Priya	65000.00
104	Sneha	70000.00

The 'HighSalaryEmployees2' tab is currently selected, showing the 'Output' section. It displays a log of actions performed during the session:

#	Time	Action	Message
1	22:13:28	show databases	10 row(s) returned
2	22:15:57	USE companydb1	0 row(s) affected
3	22:16:50	CREATE VIEW HighSalaryEmployees AS SELECT emp_id, emp_name, salary FROM employees WHERE salary...	0 row(s) affected
4	22:16:55	SELECT * FROM HighSalaryEmployees LIMIT 0, 1000	2 row(s) returned

View 4 – Employee and Department using JOIN

Question: Create a View that displays employee name, department name and salary.

DROP VIEW IF EXISTS EmployeeDepartmentView;

CREATE VIEW EmployeeDepartmentView AS

SELECT

 e.emp_id,

 e.emp_name,

 d.dept_name,

 e.salary

FROM employees e

JOIN departments d

ON e.dept_id = d.dept_id;

SELECT * FROM EmployeeDepartmentView;

Output:

The screenshot displays a SQL IDE interface. The top pane shows a query window with the following SQL code:

```
14     e.salary
15 FROM employees e
16 JOIN departments d
17 ON e.dept_id = d.dept_id;
18
19 • SELECT * FROM EmployeeDepartmentView;
20
```

The bottom pane shows the 'Result Grid' with the following data:

emp_id	emp_name	dept_name	salary
101	Rahul	CSE	50000.00
102	Priya	ECE	65000.00
103	Anil	CSE	55000.00
104	Sneha	EEE	70000.00
105	Kiran	ECE	48000.00

Below the result grid, the 'EmployeeDepartmentView 3' tab is visible. The 'Output' pane shows the 'Action Output' with the following log:

#	Time	Action	Message
1	22:13:28	show databases	10 row(s) returned
2	22:15:57	USE companydb1	0 row(s) affected
3	22:16:50	CREATE VIEW HighSalaryEmployees AS SELECT emp_id, emp_name, salary FROM employees WHERE salary...	0 row(s) affected
4	22:16:55	SELECT * FROM HighSalaryEmployees LIMIT 0, 1000	2 row(s) returned
5	22:18:41	CREATE VIEW EmployeeDepartmentView AS SELECT e.emp_id, e.emp_name, d.dept_name, e.salar...	0 row(s) affected
6	22:18:49	SELECT * FROM EmployeeDepartmentView LIMIT 0, 1000	5 row(s) returned

View 5 – Employees from a Particular Department

Question: Create a View that displays only CSE employees.

DROP VIEW IF EXISTS CSEEmployees;

CREATE VIEW CSEEmployees AS

SELECT

 e.emp_id,

 e.emp_name,

 d.dept_name,

 e.salary

FROM employees e

JOIN departments d

ON e.dept_id = d.dept_id

WHERE d.dept_name = 'CSE';

SELECT * FROM CSEEmployees;

Output:

The screenshot displays the SQL Developer interface. The top window, titled 'Query 1', contains the following SQL code:

```
26 FROM employees e
27 JOIN departments d
28 ON e.dept_id = d.dept_id
29 WHERE d.dept_name = 'CSE';
30
31 • SELECT * FROM CSEEmployees;
32
```

Below the query window is the 'Result Grid' showing the output of the query. It contains two rows of data:

emp_id	emp_name	dept_name	salary
101	Rahul	CSE	50000.00
103	Anil	CSE	55000.00

The bottom window, titled 'CSEEmployees 4', shows the 'Output' tab with a list of actions and their messages:

#	Time	Action	Message
3	22:16:50	CREATE VIEW HighSalaryEmployees AS SELECT emp_id, emp_name, salary FROM employees WHERE sala...	0 row(s) affected
4	22:16:55	SELECT * FROM HighSalaryEmployees LIMIT 0, 1000	2 row(s) returned
5	22:18:41	CREATE VIEW EmployeeDepartmentView AS SELECT e.emp_id, e.emp_name, d.dept_name, e.sal...	0 row(s) affected
6	22:18:49	SELECT * FROM EmployeeDepartmentView LIMIT 0, 1000	5 row(s) returned
7	22:20:47	CREATE VIEW CSEEmployees AS SELECT e.emp_id, e.emp_name, d.dept_name, e.salary FROM ...	0 row(s) affected
8	22:20:58	SELECT * FROM CSEEmployees LIMIT 0, 1000	2 row(s) returned

View 6 – Department-wise Average Salary

Question: Create a View using GROUP BY and AVG() to display department-wise average salary.

DROP VIEW IF EXISTS DepartmentSalaryView;

```
CREATE VIEW DepartmentSalaryView AS
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS EmployeeCount,
    AVG(e.salary) AS AverageSalary
FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;
```



```
SELECT * FROM DepartmentSalaryView;
```

Output:

The screenshot shows the SQL Developer interface. The top pane contains a SQL query:

```
37 FROM employees e
38 JOIN departments d
39 ON e.dept_id = d.dept_id
40 GROUP BY d.dept_id, d.dept_name;
41
42 • SELECT * FROM DepartmentSalaryView;
43
```

The bottom pane displays the 'Result Grid' with the following data:

dept_name	EmployeeCount	AverageSalary
CSE	2	52500.000000
ECE	2	56500.000000
EEE	1	70000.000000

Below the result grid, the 'Output' pane shows the 'Action Output' with a log of database actions:

#	Time	Action	Message
5	22:18:41	CREATE VIEW EmployeeDepartmentView AS SELECT e.emp_id, e.emp_name, d.dept_name, e.sal...	0 row(s) affected
6	22:18:49	SELECT * FROM EmployeeDepartmentView LIMIT 0, 1000	5 row(s) returned
7	22:20:47	CREATE VIEW CSEEmployees AS SELECT e.emp_id, e.emp_name, d.dept_name, e.salary FROM ...	0 row(s) affected
8	22:20:58	SELECT * FROM CSEEmployees LIMIT 0, 1000	2 row(s) returned
9	22:22:42	CREATE VIEW DepartmentSalaryView AS SELECT d.dept_name, COUNT(e.emp_id) AS EmployeeCount...	0 row(s) affected
10	22:22:51	SELECT * FROM DepartmentSalaryView LIMIT 0, 1000	3 row(s) returned

View 7 – Departments with Average Salary Above 55000

Question: Create a View that displays departments whose average salary is greater than 55000.

```
DROP VIEW IF EXISTS HighAverageDepartments;
```

```
CREATE VIEW HighAverageDepartments AS
```

```
SELECT
```

```
    d.dept_name,
```

```
    AVG(e.salary) AS AverageSalary
```

```
FROM employees e
```

```
JOIN departments d
```

```
ON e.dept_id = d.dept_id
```

```
GROUP BY d.dept_id, d.dept_name
```

```
HAVING AVG(e.salary) > 55000;
```

SELECT * FROM HighAverageDepartments;

Output:

The screenshot shows a SQL IDE interface. The top pane, titled 'Query 1', contains the following SQL code:

```
49 ON e.dept_id = d.dept_id
50 GROUP BY d.dept_id, d.dept_name
51 HAVING AVG(e.salary) > 55000;
52
53 • SELECT * FROM HighAverageDepartments;
54
55
```

The bottom pane shows the 'Result Grid' with two columns: 'dept_name' and 'AverageSalary'. It contains two rows of data:

dept_name	AverageSalary
ECE	56500.000000
EEE	70000.000000

Below the result grid, there is an 'Output' section titled 'Action Output' showing a log of database actions:

#	Time	Action	Message
7	22:20:47	CREATE VIEW CSEEmployees AS SELECT e.emp_id, e.emp_name, d.dept_name, e.salary FROM ...	0 row(s) affected
8	22:20:58	SELECT * FROM CSEEmployees LIMIT 0, 1000	2 row(s) returned
9	22:22:42	CREATE VIEW DepartmentSalaryView AS SELECT d.dept_name, COUNT(e.emp_id) AS EmployeeCount...	0 row(s) affected
10	22:22:51	SELECT * FROM DepartmentSalaryView LIMIT 0, 1000	3 row(s) returned
11	22:24:24	CREATE VIEW HighAverageDepartments AS SELECT d.dept_name, AVG(e.salary) AS AverageSalary F...	0 row(s) affected
12	22:24:47	SELECT * FROM HighAverageDepartments LIMIT 0, 1000	2 row(s) returned

View 8 – Salary Range View

Question: Create a View that displays employees with salary between 50000 and 70000.

DROP VIEW IF EXISTS SalaryRangeView;

```
CREATE VIEW SalaryRangeView AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary BETWEEN 50000 AND 70000;
```

SELECT * FROM SalaryRangeView;

Output:

The screenshot displays a database query editor with a query window and an output window.

Query Window (Query 1):

```
54 • CREATE VIEW SalaryRangeView AS
55 SELECT emp_id, emp_name, salary
56 FROM employees
57 WHERE salary BETWEEN 50000 AND 70000;
58
59 • SELECT * FROM SalaryRangeView;
60
```

Result Grid:

emp_id	emp_name	salary
101	Rahul	50000.00
102	Priya	65000.00
103	Anil	55000.00
104	Sneha	70000.00

Output Window (SalaryRangeView 7):

Output: Action Output

#	Time	Action	Message
9	22:22:42	CREATE VIEW DepartmentSalaryView AS SELECT d.dept_name, COUNT(e.emp_id) AS EmployeeCount...	0 row(s) affected
10	22:22:51	SELECT * FROM DepartmentSalaryView LIMIT 0, 1000	3 row(s) returned
11	22:24:24	CREATE VIEW HighAverageDepartments AS SELECT d.dept_name, AVG(e.salary) AS AverageSalary F...	0 row(s) affected
12	22:24:47	SELECT * FROM HighAverageDepartments LIMIT 0, 1000	2 row(s) returned
13	22:26:06	CREATE VIEW SalaryRangeView AS SELECT emp_id, emp_name, salary FROM employees WHERE salary B...	0 row(s) affected
14	22:26:16	SELECT * FROM SalaryRangeView LIMIT 0, 1000	4 row(s) returned

View 9 – View with Calculated Column

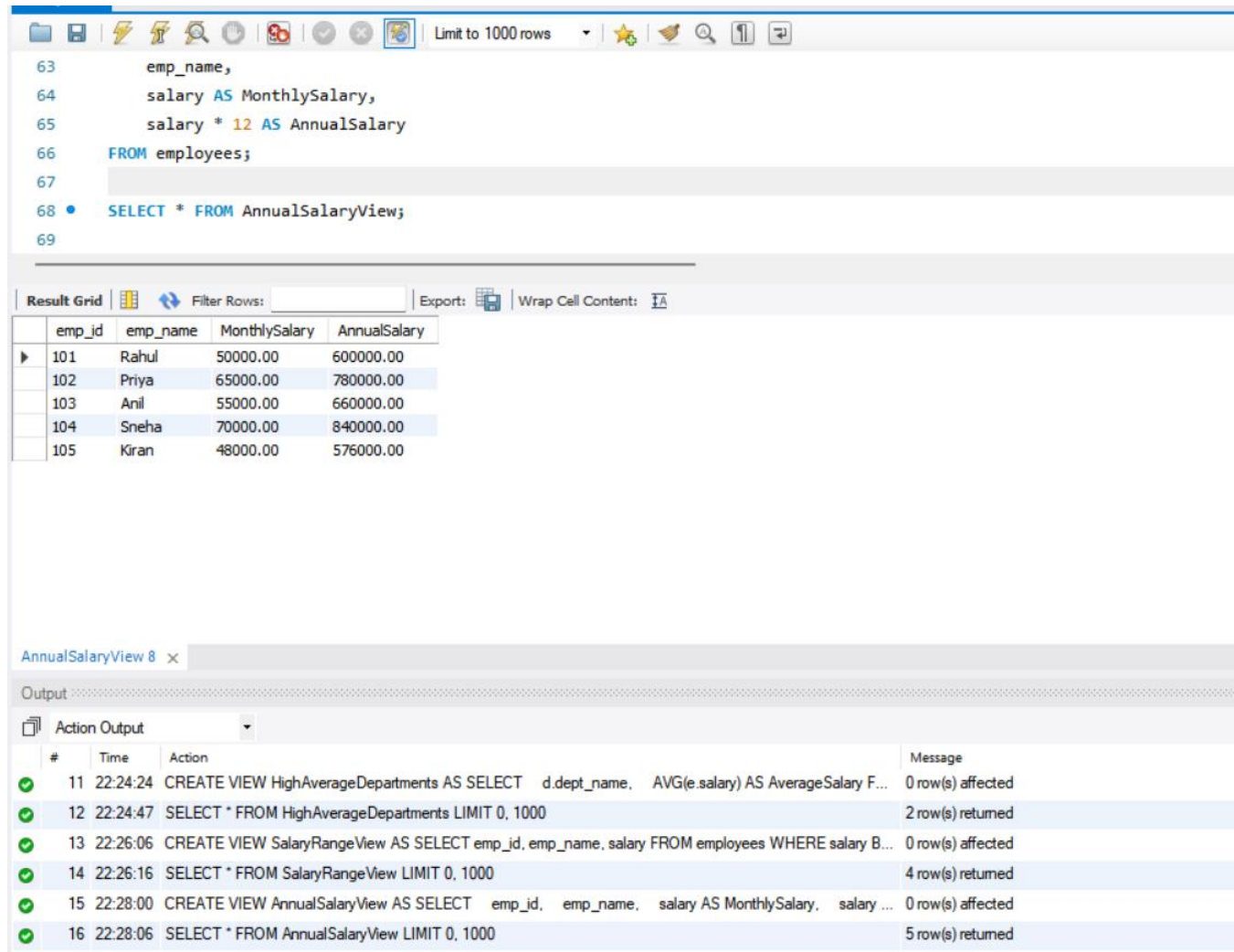
Question: Create a View that displays employee name, monthly salary and annual salary.

DROP VIEW IF EXISTS AnnualSalaryView;

```
CREATE VIEW AnnualSalaryView AS
SELECT
    emp_id,
    emp_name,
    salary AS MonthlySalary,
    salary * 12 AS AnnualSalary
FROM employees;
```

```
SELECT * FROM AnnualSalaryView;
```

Output:



The screenshot shows a SQL Developer window with a query editor and a results grid. The query editor contains the following SQL code:

```
63     emp_name,  
64     salary AS MonthlySalary,  
65     salary * 12 AS AnnualSalary  
66 FROM employees;  
67  
68 • SELECT * FROM AnnualSalaryView;  
69
```

The results grid displays the output of the query, showing 5 rows of data. The columns are emp_id, emp_name, MonthlySalary, and AnnualSalary.

emp_id	emp_name	MonthlySalary	AnnualSalary
101	Rahul	50000.00	600000.00
102	Priya	65000.00	780000.00
103	Anil	55000.00	660000.00
104	Sneha	70000.00	840000.00
105	Kiran	48000.00	576000.00

Below the results grid, the 'Output' tab is selected, showing the 'Action Output' of the SQL script. The output is as follows:

#	Time	Action	Message
11	22:24:24	CREATE VIEW HighAverageDepartments AS SELECT d.dept_name, AVG(e.salary) AS AverageSalary F...	0 row(s) affected
12	22:24:47	SELECT * FROM HighAverageDepartments LIMIT 0, 1000	2 row(s) returned
13	22:26:06	CREATE VIEW SalaryRangeView AS SELECT emp_id, emp_name, salary FROM employees WHERE salary B...	0 row(s) affected
14	22:26:16	SELECT * FROM SalaryRangeView LIMIT 0, 1000	4 row(s) returned
15	22:28:00	CREATE VIEW AnnualSalaryView AS SELECT emp_id, emp_name, salary AS MonthlySalary, salary ...	0 row(s) affected
16	22:28:06	SELECT * FROM AnnualSalaryView LIMIT 0, 1000	5 row(s) returned

View 10 – Update Data Through a Simple View

Question: Create a simple single-table View and update an employee's salary through the View.

DROP VIEW IF EXISTS EmployeeSalaryView;

```
CREATE VIEW EmployeeSalaryView AS  
SELECT emp_id, emp_name, salary  
FROM employees;
```

```
UPDATE EmployeeSalaryView  
SET salary = 60000  
WHERE emp_id = 101;
```

```
SELECT * FROM EmployeeSalaryView;  
SELECT * FROM employees WHERE emp_id = 101;
```

Output:

Query Editor

Limit to 1000 rows

```
72
73 • UPDATE EmployeeSalaryView
74   SET salary = 60000
75   WHERE emp_id = 101;
76
77 • SELECT * FROM EmployeeSalaryView;
78 • SELECT * FROM employees WHERE emp_id = 101;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	emp_id	emp_name	salary
▶	101	Rahul	60000.00
	102	Priya	65000.00
	103	Anil	55000.00
	104	Sneha	70000.00
	105	Kiran	48000.00

Query Editor

Limit to 1000 rows

```
73 • UPDATE EmployeeSalaryView
74   SET salary = 60000
75   WHERE emp_id = 101;
76
77 • SELECT * FROM EmployeeSalaryView;
78 • SELECT * FROM employees WHERE emp_id = 101;
79
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap C

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	60000.00
*	NULL	NULL	NULL	NULL

View 11 – CREATE OR REPLACE VIEW

Question: Modify an existing View so that it displays employee name and salary only.

```
CREATE OR REPLACE VIEW EmployeeBasicView AS  
SELECT emp_name, salary  
FROM employees;
```

```
SELECT * FROM EmployeeBasicView;
```

Output:

The screenshot displays the SQL Developer interface. The top pane shows the SQL script for creating and querying the view. The bottom pane shows the results of the execution, including a table of employee names and salaries, and a log of the executed statements.

SQL Script:

```
79  
80 • CREATE OR REPLACE VIEW EmployeeBasicView AS  
81   SELECT emp_name, salary  
82   FROM employees;  
83  
84 • SELECT * FROM EmployeeBasicView;  
85
```

Result Grid:

emp_name	salary
Rahul	60000.00
Priya	65000.00
Anil	55000.00
Sneha	70000.00
Kiran	48000.00

EmployeeBasicView 11

Output:

#	Time	Action	Message
✓ 17	22:29:42	CREATE VIEW EmployeeSalaryView AS SELECT emp_id, emp_name, salary FROM employees	0 row(s) affected
✓ 18	22:29:42	UPDATE EmployeeSalaryView SET salary = 60000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 19	22:29:50	SELECT * FROM EmployeeSalaryView LIMIT 0, 1000	5 row(s) returned
✓ 20	22:30:41	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
✓ 21	22:31:44	CREATE OR REPLACE VIEW EmployeeBasicView AS SELECT emp_name, salary FROM employees	0 row(s) affected
✓ 22	22:31:53	SELECT * FROM EmployeeBasicView LIMIT 0, 1000	5 row(s) returned

View 12 – View Definition

Question: Display the SQL statement used to create a View.

```
SHOW CREATE VIEW EmployeeDepartmentView;
```

Output:

Query 1 x

Limit to 1000 rows

```

80 • CREATE OR REPLACE VIEW EmployeeBasicView AS
81   SELECT emp_name, salary
82   FROM employees;
83
84 • SELECT * FROM EmployeeBasicView;
85 • SHOW CREATE VIEW EmployeeDepartmentView;
86

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [f](#)

View	Create View	character_set_client	collation_connection
employeedepartmentview	CREATE ALGORITHM=UNDEFINED DEFINER='r...' utf8mb4		utf8mb4_0900_ai_ci

Result 12 x Read Only

Output

Action Output

#	Time	Action	Message
✓ 18	22:29:42	UPDATE EmployeeSalaryView SET salary = 60000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 19	22:29:50	SELECT * FROM EmployeeSalaryView LIMIT 0, 1000	5 row(s) returned
✓ 20	22:30:41	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
✓ 21	22:31:44	CREATE OR REPLACE VIEW EmployeeBasicView AS SELECT emp_name, salary FROM employees	0 row(s) affected
✓ 22	22:31:53	SELECT * FROM EmployeeBasicView LIMIT 0, 1000	5 row(s) returned
✓ 23	22:33:39	SHOW CREATE VIEW EmployeeDepartmentView	1 row(s) returned

View 13 – List All Views

Question: Display all Views available in the current database.

SHOW FULL TABLES

WHERE Table_type = 'VIEW';

Output:

The screenshot displays the SQL Server Enterprise Manager interface. At the top, a query window titled 'Query 1' shows the following SQL code:

```
82 FROM employees;
83
84 • SELECT * FROM EmployeeBasicView;
85 • SHOW CREATE VIEW EmployeeDepartmentView;
86 • SHOW FULL TABLES
87 WHERE Table_type = 'VIEW';
88
```

Below the query window, the 'Result Grid' tab is active, showing a table with two columns: 'Tables_in_companydb1' and 'Table_type'. The table contains 10 rows, all of which are views.

Tables_in_companydb1	Table_type
annualsalaryview	VIEW
csemployees	VIEW
departmentsalaryview	VIEW
employeebasicview	VIEW
employeeedepartmentview	VIEW
employeesalaryview	VIEW
employeeview	VIEW
highaveragedepartments	VIEW
highsalaryemployees	VIEW
salaryrangeview	VIEW

Below the result grid, the 'Output' tab is active, showing a table with four columns: '#', 'Time', 'Action', and 'Message'. The table contains 6 rows of execution results.

#	Time	Action	Message
19	22:29:50	SELECT * FROM EmployeeSalaryView LIMIT 0, 1000	5 row(s) returned
20	22:30:41	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
21	22:31:44	CREATE OR REPLACE VIEW EmployeeBasicView AS SELECT emp_name, salary FROM employees	0 row(s) affected
22	22:31:53	SELECT * FROM EmployeeBasicView LIMIT 0, 1000	5 row(s) returned
23	22:33:39	SHOW CREATE VIEW EmployeeDepartmentView	1 row(s) returned
24	22:35:16	SHOW FULL TABLES WHERE Table_type = 'VIEW'	10 row(s) returned

View 14 – Drop a View

Question: Remove an existing View from the database.

DROP VIEW IF EXISTS SalaryRangeView;

4. Important Concept: Simple View vs Complex View

A simple View is generally based on one table and does not use grouping or aggregate functions. Some simple Views can be updated. A View containing JOINS, GROUP BY, aggregate functions such as AVG(), DISTINCT, or other non-updatable constructs may not be directly updatable.

5. View vs Table

Feature	Table	View
Stores data	Yes	Normally no; stores query definition

Created with CREATE TABLE	Yes	No
Created with CREATE VIEW	No	Yes
Can use JOINS	Not applicable	Yes
Reflects current underlying data	Own stored data	Yes
Can always be updated	Yes	No; depends on the View

6. Student Practice Tasks

1. Create a View showing only emp_id, emp_name and salary.

Output:

Query 1

```

87 WHERE Table_type = 'VIEW';
88 • DROP VIEW IF EXISTS SalaryRangeView;
89 • CREATE VIEW employee_view AS
90 SELECT emp_id, emp_name, salary
91 FROM employees;
92 • SELECT * FROM employee_view;
93

```

Limit to 1000 rows

Result Grid

emp_id	emp_name	salary
101	Rahul	60000.00
102	Priya	65000.00
103	Anil	55000.00
104	Sneha	70000.00
105	Kiran	48000.00

employee_view 14 x

Output

Action Output

#	Time	Action	Message
✓ 22	22:31:53	SELECT * FROM EmployeeBasicView LIMIT 0, 1000	5 row(s) returned
✓ 23	22:33:39	SHOW CREATE VIEW EmployeeDepartmentView	1 row(s) returned
✓ 24	22:35:16	SHOW FULL TABLES WHERE Table_type = 'VIEW'	10 row(s) returned
✓ 25	22:37:00	DROP VIEW IF EXISTS SalaryRangeView	0 row(s) affected
✓ 26	22:40:57	CREATE VIEW employee_view AS SELECT emp_id, emp_name, salary FROM employees	0 row(s) affected
✓ 27	22:41:19	SELECT * FROM employee_view LIMIT 0, 1000	5 row(s) returned

2. Create a View for employees earning more than 60000.

Output:

The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following code:

```
92 • SELECT * FROM employee_view;
93 • CREATE VIEW high_salary_employees AS
94   SELECT *
95   FROM employees
96   WHERE salary > 60000;
97 • SELECT * FROM high_salary_employees;
98
```

Below the editor is the 'Result Grid' section, which displays the results of the last query. It includes a 'Filter Rows' input and an 'Export' button. The results are as follows:

	emp_id	emp_name	dept_id	salary
▶	102	Priya	2	65000.00
	104	Sneha	3	70000.00

At the bottom, the 'Output' pane shows the 'Action Output' log. It contains the following entries:

#	Time	Action	Message
✓ 24	22:35:16	SHOW FULL TABLES WHERE Table_type = 'VIEW'	10 row(s) returned
✓ 25	22:37:00	DROP VIEW IF EXISTS SalaryRangeView	0 row(s) affected
✓ 26	22:40:57	CREATE VIEW employee_view AS SELECT emp_id, emp_name, salary FROM employees	0 row(s) affected
✓ 27	22:41:19	SELECT * FROM employee_view LIMIT 0, 1000	5 row(s) returned
✓ 28	22:43:59	CREATE VIEW high_salary_employees AS SELECT * FROM employees WHERE salary > 60000	0 row(s) affected
✓ 29	22:44:23	SELECT * FROM high_salary_employees LIMIT 0, 1000	2 row(s) returned

3. Create a View showing employee name, department name and salary using JOIN.

Output:

The screenshot displays a database query editor interface. The top section shows a SQL script for creating a view and querying it. The script is as follows:

```
98 • CREATE VIEW employee_department_view AS
99   SELECT e.emp_name, d.dept_name, e.salary
100  FROM employees e
101  JOIN departments d
102  ON e.dept_id = d.dept_id;
103 • SELECT * FROM employee_department_view;
104
```

Below the script, the 'Result Grid' is visible, showing the output of the query. The grid has three columns: emp_name, dept_name, and salary. The data is as follows:

emp_name	dept_name	salary
Rahul	CSE	60000.00
Priya	ECE	65000.00
Anil	CSE	55000.00
Sneha	EEE	70000.00
Kiran	ECE	48000.00

At the bottom of the interface, the 'Action Output' section shows a log of database actions and their results:

#	Time	Action	Message
26	22:40:57	CREATE VIEW employee_view AS SELECT emp_id, emp_name, salary FROM employees	0 row(s) affected
27	22:41:19	SELECT * FROM employee_view LIMIT 0, 1000	5 row(s) returned
28	22:43:59	CREATE VIEW high_salary_employees AS SELECT * FROM employees WHERE salary > 60000	0 row(s) affected
29	22:44:23	SELECT * FROM high_salary_employees LIMIT 0, 1000	2 row(s) returned
30	22:46:04	CREATE VIEW employee_department_view AS SELECT e.emp_name, d.dept_name, e.salary FROM employee...	0 row(s) affected
31	22:46:26	SELECT * FROM employee_department_view LIMIT 0, 1000	5 row(s) returned

4. Create a View showing only ECE employees.

Output:

The screenshot displays a database query editor interface. The top section shows a SQL query for 'Query 1' with line numbers 107 to 113. The query is as follows:

```
107 WHERE dept_id = (  
108     SELECT dept_id  
109     FROM departments  
110     WHERE dept_name = 'ECE'  
111 );  
112 • SELECT * FROM ece_employees;  
113
```

Below the query editor, the 'Result Grid' is visible, showing two rows of data:

emp_id	emp_name	dept_id	salary
102	Priya	2	65000.00
105	Kiran	2	48000.00

The bottom section of the interface shows the 'Output' tab for 'ece_employees 17'. It contains an 'Action Output' table with columns for line number, time, action, and message.

#	Time	Action	Message
✓ 28	22:43:59	CREATE VIEW high_salary_employees AS SELECT * FROM employees WHERE salary > 60000	0 row(s) affected
✓ 29	22:44:23	SELECT * FROM high_salary_employees LIMIT 0, 1000	2 row(s) returned
✓ 30	22:46:04	CREATE VIEW employee_department_view AS SELECT e.emp_name, d.dept_name, e.salary FROM employe...	0 row(s) affected
✓ 31	22:46:26	SELECT * FROM employee_department_view LIMIT 0, 1000	5 row(s) returned
✓ 32	22:49:45	CREATE VIEW ece_employees AS SELECT * FROM employees WHERE dept_id = (SELECT dept_id F...	0 row(s) affected
✓ 33	22:50:06	SELECT * FROM ece_employees LIMIT 0, 1000	2 row(s) returned

5. Create a View showing department-wise employee count.

Output:

The screenshot displays a database IDE interface. The top section shows a SQL script with the following queries:

```
114 SELECT d.dept_name, COUNT(e.emp_id) AS employee_count
115 FROM departments d
116 LEFT JOIN employees e
117 ON d.dept_id = e.dept_id
118 GROUP BY d.dept_id, d.dept_name;
119 • SELECT * FROM department_employee_count;
120
```

Below the script, the 'Result Grid' shows the output of the first query:

dept_name	employee_count
CSE	2
ECE	2
EEE	1

The bottom section, labeled 'Output', shows the 'Action Output' log with the following entries:

#	Time	Action	Message
30	22:46:04	CREATE VIEW employee_department_view AS SELECT e.emp_name, d.dept_name, e.salary FROM employee...	0 row(s) affected
31	22:46:26	SELECT * FROM employee_department_view LIMIT 0, 1000	5 row(s) returned
32	22:49:45	CREATE VIEW ece_employees AS SELECT * FROM employees WHERE dept_id = (SELECT dept_id F...	0 row(s) affected
33	22:50:06	SELECT * FROM ece_employees LIMIT 0, 1000	2 row(s) returned
34	22:52:32	CREATE VIEW department_employee_count AS SELECT d.dept_name, COUNT(e.emp_id) AS employee_cou...	0 row(s) affected
35	22:52:56	SELECT * FROM department_employee_count LIMIT 0, 1000	3 row(s) returned

6. Create a View showing department-wise maximum salary.

Output:

The screenshot displays a SQL IDE interface. The top section shows a query editor with the following SQL code:

```
121 SELECT d.dept_name, MAX(e.salary) AS max_salary
122 FROM departments d
123 JOIN employees e
124 ON d.dept_id = e.dept_id
125 GROUP BY d.dept_id, d.dept_name;
126 • SELECT * FROM department_max_salary;
127
```

Below the query editor, the 'Result Grid' tab is active, showing the output of the query. The table has two columns: 'dept_name' and 'max_salary'.

dept_name	max_salary
CSE	60000.00
ECE	65000.00
EEE	70000.00

At the bottom, the 'Action Output' tab is active, showing a log of database actions. The log includes the following entries:

#	Time	Action	Message
32	22:49:45	CREATE VIEW ece_employees AS SELECT * FROM employees WHERE dept_id = (SELECT dept_id F...	0 row(s) affected
33	22:50:06	SELECT * FROM ece_employees LIMIT 0, 1000	2 row(s) returned
34	22:52:32	CREATE VIEW department_employee_count AS SELECT d.dept_name, COUNT(e.emp_id) AS employee_cou...	0 row(s) affected
35	22:52:56	SELECT * FROM department_employee_count LIMIT 0, 1000	3 row(s) returned
36	22:54:40	CREATE VIEW department_max_salary AS SELECT d.dept_name, MAX(e.salary) AS max_salary FROM depar...	0 row(s) affected
37	22:55:07	SELECT * FROM department_max_salary LIMIT 0, 1000	3 row(s) returned

7. Create a View showing departments whose average salary is greater than 50000.

Output:

Query 1

```
129 FROM departments d
130 JOIN employees e
131 ON d.dept_id = e.dept_id
132 GROUP BY d.dept_id, d.dept_name
133 HAVING AVG(e.salary) > 50000;
134 • SELECT * FROM high_avg_salary_departments;
135
```

Result Grid

	dept_name	average_salary
▶	CSE	57500.000000
	ECE	56500.000000
	EEE	70000.000000

high_avg_salary_departments20

Output

Action Output

#	Time	Action	Message
✓ 34	22:52:32	CREATE VIEW department_employee_count AS SELECT d.dept_name, COUNT(e.emp_id) AS employee_cou...	0 row(s) affected
✓ 35	22:52:56	SELECT * FROM department_employee_count LIMIT 0, 1000	3 row(s) returned
✓ 36	22:54:40	CREATE VIEW department_max_salary AS SELECT d.dept_name, MAX(e.salary) AS max_salary FROM depar...	0 row(s) affected
✓ 37	22:55:07	SELECT * FROM department_max_salary LIMIT 0, 1000	3 row(s) returned
✓ 38	22:58:34	CREATE VIEW high_avg_salary_departments AS SELECT d.dept_name, AVG(e.salary) AS average_salary F...	0 row(s) affected
✓ 39	22:58:55	SELECT * FROM high_avg_salary_departments LIMIT 0, 1000	3 row(s) returned

8. Create a View with employee name, salary and annual salary.

Output:

The screenshot displays a database IDE interface. The top section shows a SQL script with the following lines:

```
133 HAVING AVG(e.salary) > 50000;  
134 • SELECT * FROM high_avg_salary_departments;  
135 • CREATE VIEW employee_annual_salary AS  
136 SELECT emp_name, salary, salary * 12 AS annual_salary  
137 FROM employees;  
138 • SELECT * FROM employee_annual_salary;  
139
```

Below the script, the 'Result Grid' shows the output of the last query. It contains a table with three columns: emp_name, salary, and annual_salary. The data is as follows:

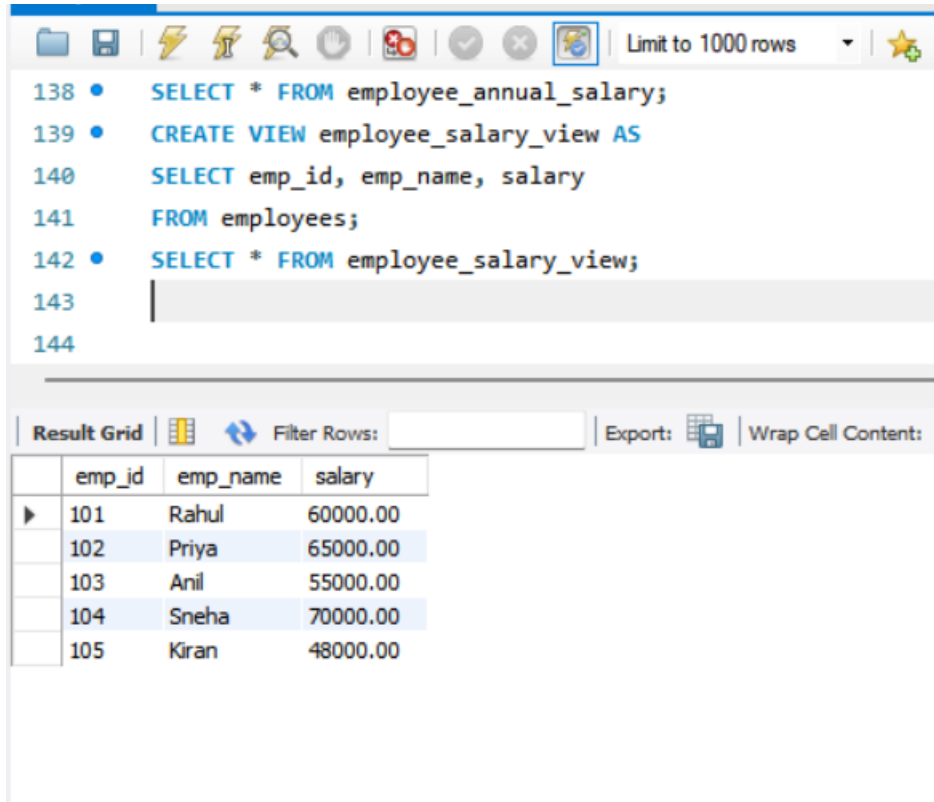
emp_name	salary	annual_salary
Rahul	60000.00	720000.00
Priya	65000.00	780000.00
Anil	55000.00	660000.00
Sneha	70000.00	840000.00
Kiran	48000.00	576000.00

The bottom section of the IDE shows the 'Action Output' log, which records the execution of the SQL statements:

#	Time	Action	Message
36	22:54:40	CREATE VIEW department_max_salary AS SELECT d.dept_name, MAX(e.salary) AS max_salary FROM depar...	0 row(s) affected
37	22:55:07	SELECT * FROM department_max_salary LIMIT 0, 1000	3 row(s) returned
38	22:58:34	CREATE VIEW high_avg_salary_departments AS SELECT d.dept_name, AVG(e.salary) AS average_salary F...	0 row(s) affected
39	22:58:55	SELECT * FROM high_avg_salary_departments LIMIT 0, 1000	3 row(s) returned
40	23:01:25	CREATE VIEW employee_annual_salary AS SELECT emp_name, salary, salary * 12 AS annual_salary FROM ...	0 row(s) affected
41	23:01:49	SELECT * FROM employee_annual_salary LIMIT 0, 1000	5 row(s) returned

9. Update an employee's salary through a simple View and verify the base table.

Output:



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following queries:

```
138 • SELECT * FROM employee_annual_salary;  
139 • CREATE VIEW employee_salary_view AS  
140     SELECT emp_id, emp_name, salary  
141     FROM employees;  
142 • SELECT * FROM employee_salary_view;  
143  
144
```

Below the editor is the 'Result Grid' section, which includes a 'Filter Rows' input and an 'Export' button. The result grid displays the output of the last query, showing a table with 5 rows and 3 columns: emp_id, emp_name, and salary.

	emp_id	emp_name	salary
▶	101	Rahul	60000.00
	102	Priya	65000.00
	103	Anil	55000.00
	104	Sneha	70000.00
	105	Kiran	48000.00

Limit to 1000 rows

```
142 • SELECT * FROM employee_salary_view;
143 • UPDATE employee_salary_view
144     SET salary = 65000
145     WHERE emp_id = 101;
146 • SELECT * FROM employee_salary_view
147     WHERE emp_id = 101;
148
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

	emp_id	emp_name	salary
▶	101	Rahul	65000.00

Query 1 x

Limit to 1000 rows

```
146 • SELECT * FROM employee_salary_view
147     WHERE emp_id = 101;
148 • SELECT emp_id, emp_name, salary
149     FROM employees
150     WHERE emp_id = 101;
151
152
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: [A](#)

	emp_id	emp_name	salary
▶	101	Rahul	65000.00
*	NULL	NULL	NULL

10. Use SHOW CREATE VIEW to display the definition of a View.

Output:

The screenshot displays a database query tool interface. The top section shows a SQL query editor with the following queries:

```
146 • SELECT * FROM employee_salary_view
147 WHERE emp_id = 101;
148 • SELECT emp_id, emp_name, salary
149 FROM employees
150 WHERE emp_id = 101;
151 • SHOW CREATE VIEW employee_salary_view;
152
```

Below the query editor is a 'Result Grid' section. It contains a table with the following data:

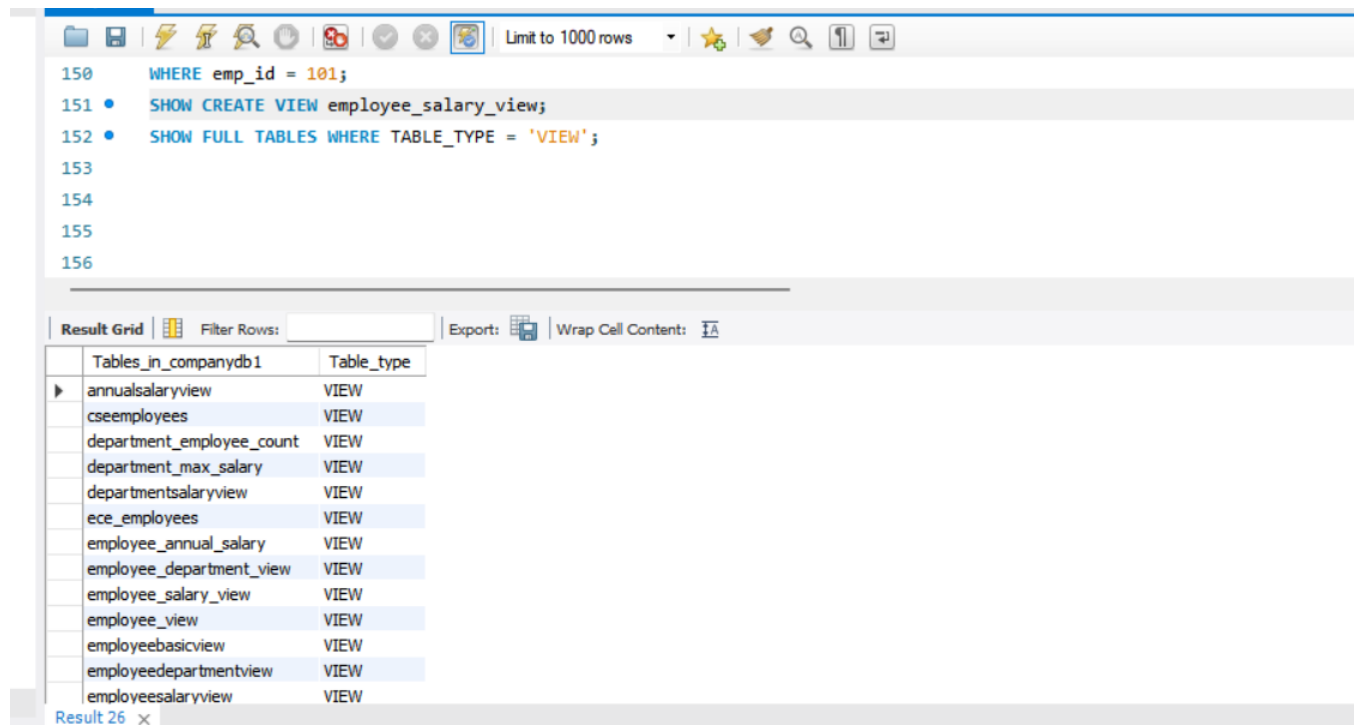
View	Create View	character_set_client	collation_connection
employee_salary_view	CREATE ALGORITHM=UNDEFINED DEFINER='r...'	utf8mb4	utf8mb4_0900_ai_ci

The bottom section shows the 'Output' of the queries, with a table listing the actions and their results:

#	Time	Action	Message
42	23:05:09	CREATE VIEW employee_salary_view AS SELECT emp_id, emp_name, salary FROM employees	0 row(s) affected
43	23:05:37	SELECT * FROM employee_salary_view LIMIT 0, 1000	5 row(s) returned
44	23:07:05	UPDATE employee_salary_view SET salary = 65000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
45	23:07:26	SELECT * FROM employee_salary_view WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
46	23:09:22	SELECT emp_id, emp_name, salary FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
47	23:20:09	SHOW CREATE VIEW employee_salary_view	1 row(s) returned

11. List all Views in the database.

Output:



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and search. The query editor contains the following SQL code:

```
150 WHERE emp_id = 101;
151 • SHOW CREATE VIEW employee_salary_view;
152 • SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW';
153
154
155
156
```

Below the query editor is the 'Result Grid' section. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The results are displayed in a table with two columns: 'Tables_in_companydb1' and 'Table_type'.

Tables_in_companydb1	Table_type
▶ annualsalaryview	VIEW
csemployees	VIEW
department_employee_count	VIEW
department_max_salary	VIEW
departmentsalaryview	VIEW
ece_employees	VIEW
employee_annual_salary	VIEW
employee_department_view	VIEW
employee_salary_view	VIEW
employee_view	VIEW
employeebasicview	VIEW
employeeedepartmentview	VIEW
employeesalaryview	VIEW

At the bottom left, there is a tab labeled 'Result 26' with a close button (x).

12. Drop a View and verify that it no longer appears in the list.

Output:

The screenshot shows a SQL IDE interface. The top window, titled 'Query 1', contains the following SQL code:

```
150 WHERE emp_id = 101;
151 • SHOW CREATE VIEW employee_salary_view;
152 • SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW';
153 • DROP VIEW employee_salary_view;
154 • SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW';
155
156
```

Below the query window is a 'Result Grid' tab. It displays a table with two columns: 'Tables_in_companydb1' and 'Table_type'. The table lists 15 views, all of which are of type 'VIEW'. The view 'employee_salary_view' is not present in the list, confirming its successful deletion.

Tables_in_companydb1	Table_type
annualsalaryview	VIEW
cseemployees	VIEW
department_employee_count	VIEW
department_max_salary	VIEW
departmentsalaryview	VIEW
ece_employees	VIEW
employee_annual_salary	VIEW
employee_department_view	VIEW
employee_view	VIEW
employeebasicview	VIEW
employeedepartmentview	VIEW
employeesalaryview	VIEW
employeeview	VIEW

The bottom of the interface shows a tab labeled 'Result 27'.